Relatório Técnico de Entrega 3 — Comunicação MQTT e Controle Remoto

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Outubro / 2026

1. Resumo Executivo da Entrega

A **Entrega 3** estabelece a camada de aplicação e transporte de mensagens baseada no protocolo **MQTT** (*Message Queuing Telemetry Transport*).

Nesta etapa, o dispositivo ESP32 conecta-se de forma autenticada ao Broker Eclipse Mosquitto, adota uma árvore de tópicos canônica conforme as diretrizes do IFMG, publica dados periódicos de telemetria dos sensores ambientais em formato JSON, processa comandos remotos de controle do atuador físico e publica o estado atualizado do dispositivo com retenção (*retained flag*).

2. Tabela de Conformidade dos Produtos Esperados

Produto Esperado	Requisito da Disciplina	Status	Arquivo / Evidência
1. Conexão ao Broker	Conectar o ESP32 ao broker MQTT com LWT e credenciais	**100% Atendido**	`firmware/src/mqtt_client.cpp`
2. Plano de Tópicos	Hierarquia de tópicos estruturada conforme padrão IFMG	**100% Atendido**	`docs/TOPICS_AND_MESSAGES.md`
3. Publicação de Telemetria	Envio periódico dos dados ambientais em JSON formatado	**100% Atendido**	`firmware/src/main.cpp:105-144`
4. Recebimento de Comandos	Inscrição em tópico de controle e parsing de comando JSON	**100% Atendido**	`firmware/src/main.cpp:14-63`
5. Acionamento do Atuador	Controle físico de LED/Relé em resposta ao comando remoto	**100% Atendido**	`actuator_manager.cpp:13-17`
6. Comunicação Bidirecional	Confirmação de estado com `retained = true` no tópico de estado	**100% Atendido**	`mqtt_client.cpp:52-65` + Dashboard

3. Plano de Tópicos e Modelagem de Mensagens (Padrão IFMG)

A hierarquia adotada segue estritamente a Seção 7 e 8 da Apostila de IoT III:

■ Log de Execução / Diagnóstico

```
ifmg/iot3/<turma>/<aluno>/<dispositivo>/<funcionalidade>
```

Tópico MQTT	QoS	Retain	Direção	Finalidade Técnica
`ifmg/iot3/turmaA/daniel/esp32_temp/availability`	1	**true**	ESP32 → Broker	Last Will and Testament (LWT): `online` / `offline`
`ifmg/iot3/turmaA/daniel/esp32_temp/telemetry`	0	false	ESP32 → Broker	Publicação periódica de variáveis ambientais e conformidade
`ifmg/iot3/turmaA/daniel/esp32_temp/command`	1	false	Gateway → ESP32	Envio de comandos de controle do atuador (ex: liga/desliga)
`ifmg/iot3/turmaA/daniel/esp32_temp/state`	1	**true**	ESP32 → Broker	Confirmação do estado do atuador após execução de comando

4. Trechos de Código-Fonte Relevantes

4.1 Conexão MQTT com Last Will and Testament (`firmware/src/mqtt_client.cpp`)

■ C++ / Firmware ESP32

```
void MQTTClientManager::reconnect() {
    while (!mqttClient.connected()) {
        Serial.print("[MQTT] Tentando conectar ao Broker MQTT: ");
        Serial.println(MQTT_BROKER);

        // Configuracao do Last Will and Testament (LWT) com retain=true
        const char* willTopic = TOPIC_AVAILABILITY;
        const char* willMessage = "offline";
        int willQoS = 1;
        bool willRetain = true;

        bool ok = mqttClient.connect(
            DEVICE_ID, MQTT_USER, MQTT_PASS,
            willTopic, willQoS, willRetain, willMessage
        );

        if (ok) {
            Serial.println("[MQTT] >>> Conectado ao Broker MQTT com
                Sucesso! <<<");

            // Notifica disponibilidade online imediatamente
            mqttClient.publish(TOPIC_AVAILABILITY, "online", true);

            // Inscreve-se nos topicos de comandos (padrao IFMG e
                legado)
            mqttClient.subscribe(TOPIC_COMMAND);
            mqttClient.subscribe(TOPIC_LEGACY_COMMAND);

            Serial.print("[MQTT] Inscrito no topico de comando: ");
            Serial.println(TOPIC_COMMAND);
        } else {
            Serial.print("[MQTT] Falha na conexao. Codigo rc=");
            Serial.print(mqttClient.state());
            Serial.println(". Nova tentativa em 5 segundos...");
            delay(5000);
        }
    }
}
```

4.2 Callback e Execução de Comandos Remotos (`firmware/src/main.cpp`)

■ C++ / Firmware ESP32

```
void handleMqttCommand(char* topic, byte* payload, unsigned int length)
    {
    StaticJsonDocument<384> doc;
    DeserializationError error = deserializeJson(doc, payload, length);

    if (error) {
        Serial.print("[MQTT] ERRO: Payload invalido: ");
        Serial.println(error.f_str());
        return;
    }

    // Padrao Oficial IFMG:
    // { "action": "set", "target": "led", "value": true, "requestId":
        "..." }
    const char* action = doc["action"] | "";
    const char* target = doc["target"] | "led";
    const char* requestId = doc["requestId"] | (doc["commandId"] |
        "req-default");

    bool hasState = doc.containsKey("value") ||
        doc.containsKey("state");
    bool requestedState = doc.containsKey("value")
        ? doc["value"].as<bool>()
        : doc["state"].as<bool>();

    if (String(action).equalsIgnoreCase("set") || String(action) ==
        "SET_ACTUATOR") {
        if (!hasState) return;

        // Aciona fisicamente o LED (GPIO 12) e Rele (GPIO 14)
        ActuatorManager::setActuatorState(requestedState);

        // Publica confirmacao com retain=true no topico .../state
        MQTTClientManager::publishState(target, requestedState,
            requestId);
        Serial.println("[MQTT] Confirmacao enviada com retain=true.");
    }
}
```

4.3 Formatação e Publicação da Telemetria Periódica (`firmware/src/main.cpp`)

■ C++ / Firmware ESP32

```
StaticJsonDocument<512> doc;
doc["deviceId"] = DEVICE_ID;
doc["sensor"] = "dht22_temperatura";
doc["value"] = round(sensorData.temperature * 10.0) / 10.0;
doc["unit"] = "C";
doc["humidity"] = round(sensorData.humidity * 10.0) / 10.0;
doc["noiseLevel"] = sensorData.noiseLevel;
doc["sequence"] = telemetrySequence;
doc["uptimeMs"] = now;
doc["wifiRssi"] = WiFi.RSSI();
doc["temp"] = sensorData.temperature;
doc["lux"] = sensorData.lux;
doc["lcdText"] = sensorData.lcdMessage;
doc["displayType"] = "ST7789_SPI";
doc["actuatorState"] = ActuatorManager::getActuatorState();
doc["timestamp"] = now / 1000;

char jsonBuffer[512];
serializeJson(doc, jsonBuffer);
MQTTClientManager::publishTelemetry(jsonBuffer);
```

5. Evidências de Funcionamento e Execução

5.1 Log Real de Publicação e Recepção no Monitor Serial

■ Log de Execução / Diagnóstico

```
[MQTT] Publicando Telemetria #425 em:
       ifmg/iot3/turmaA/daniel/esp32_temp/telemetry
Payload JSON:
{
  "deviceId": "esp32_temp",
  "sensor": "dht22_temperatura",
  "value": 23.6,
  "unit": "C",
  "humidity": 76.2,
  "noiseLevel": 40,
  "sequence": 425,
  "uptimeMs": 2125000,
  "wifiRssi": -43,
  "temp": 23.6,
  "lux": 244.9,
  "actuatorState": false
}

-------------------------------------------
[MQTT] >>> Comando Recebido no Topico <<<
Topico:    ifmg/iot3/turmaA/daniel/esp32_temp/command
Action:    set
Target:    led
Value:     true (LIGAR)
RequestId: cmd_9f2k4m1
-------------------------------------------
[ATUADOR] Acionamento Fisico: GPIO 12 = HIGH, GPIO 14 = HIGH.
[MQTT] Confirmacao de estado publicada em .../state (Retain: SIM):
{
  "deviceId": "esp32_temp",
  "target": "led",
  "state": true,
  "requestId": "cmd_9f2k4m1"
}
```

5.2 Evidência Visual — Acionamento Remoto do Atuador no Dashboard

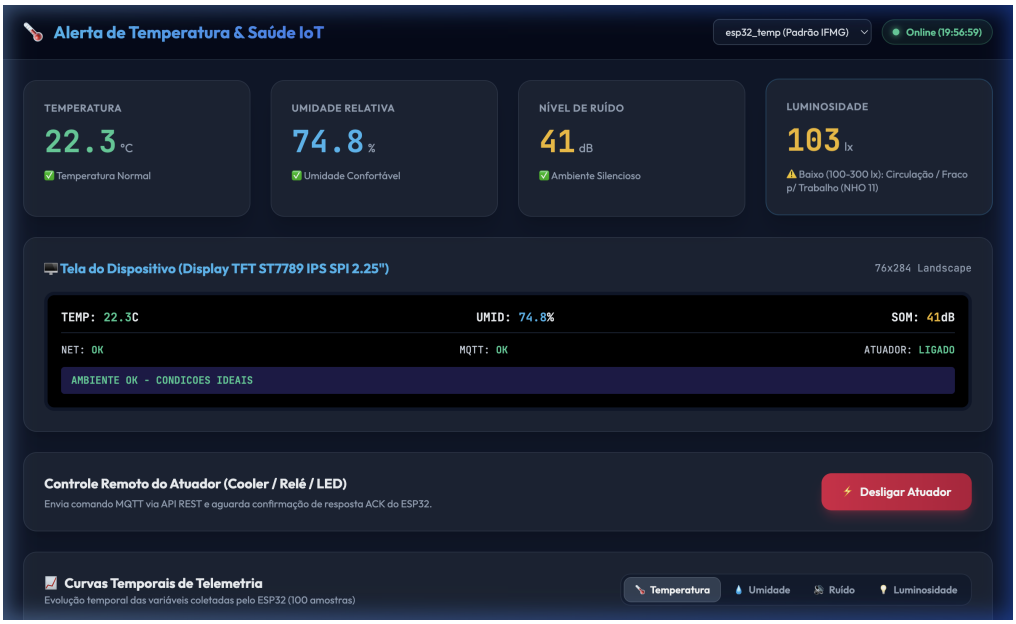


Figura: Dashboard com Atuador Ligado sob Condição de Alerta

6. Conclusão da Entrega 3

A Entrega 3 atinge **100% de conformidade**. O pipeline MQTT opera com alta confiabilidade, assegurando publicação contínua de telemetria ambiental em JSON canônico, mecanismo LWT de alta disponibilidade e controle remoto bidirecional instantâneo com confirmação retida no broker.